qwen3.5-397b-a17b-hf-model-card

---

library_name: transformers license: apache-2.0 license_link: https://huggingface.co/Qwen/Qwen3.5-397B-A17B/blob/main/LICENSE pipeline_tag: image-text-to-text

Qwen3.5-397B-A17B





> [!Note] This repository contains model weights and configuration files for the post-trained model in the Hugging Face Transformers format.
>
> These artifacts are compatible with Hugging Face Transformers, vLLM, SGLang, KTransformers, etc.
>
> [!Tip] For users seeking managed, scalable inference without infrastructure maintenance, the official Qwen API service is provided by Alibaba Cloud Model Studio.
>
> In particular, **Qwen3.5-Plus** is the hosted version corresponding to Qwen3.5-397B-A17B with more production features, e.g., 1M context length by default, official built-in tools, and adaptive tool use. For more information, please refer to the User Guide.

Over recent months, we have intensified our focus on developing foundation models that deliver exceptional utility and performance. Qwen3.5 represents a significant leap forward, integrating breakthroughs in multimodal learning, architectural efficiency, reinforcement learning scale, and global accessibility to empower developers and enterprises with unprecedented capability and efficiency.

Qwen3.5 Highlights

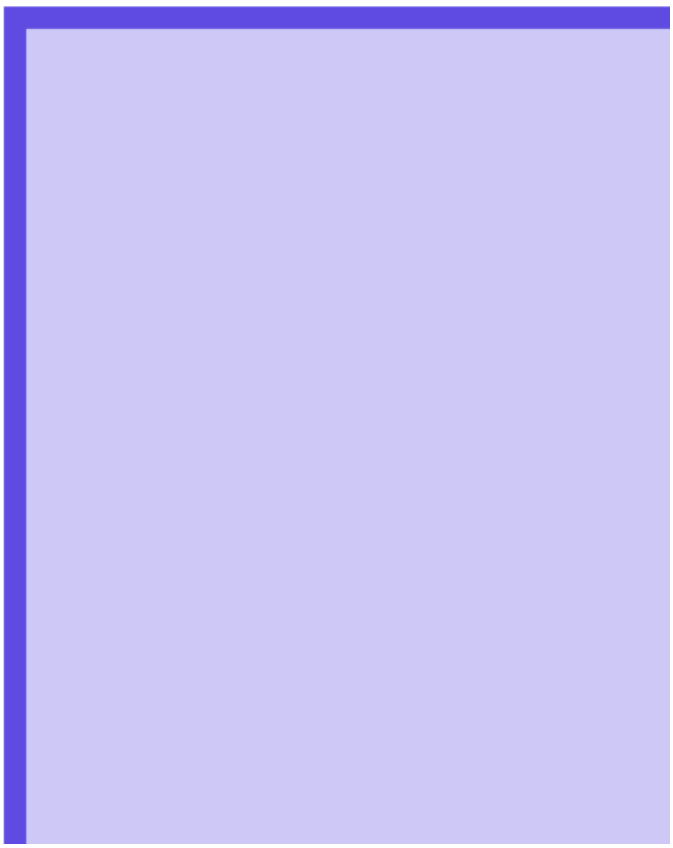
Qwen3.5 features the following enhancement:

- **Unified Vision-Language Foundation**: Early fusion training on multimodal tokens achieves cross-generational parity with Qwen3 and outperforms Qwen3-VL models across reasoning, coding, agents, and visual understanding benchmarks.
- **Efficient Hybrid Architecture**: Gated Delta Networks combined with sparse Mixture-of-Experts deliver high-throughput inference with minimal latency and cost overhead.
- **Scalable RL Generalization**: Reinforcement learning scaled across million-agent environments with progressively complex task distributions for robust real-world adaptability.
- **Global Linguistic Coverage**: Expanded support to 201 languages and dialects, enabling inclusive, worldwide deployment with nuanced cultural and regional understanding.
- **Next-Generation Training Infrastructure**: Near-100% multimodal training efficiency compared to text-only training and asynchronous RL frameworks supporting massive-scale agent scaffolds and environment orchestration.

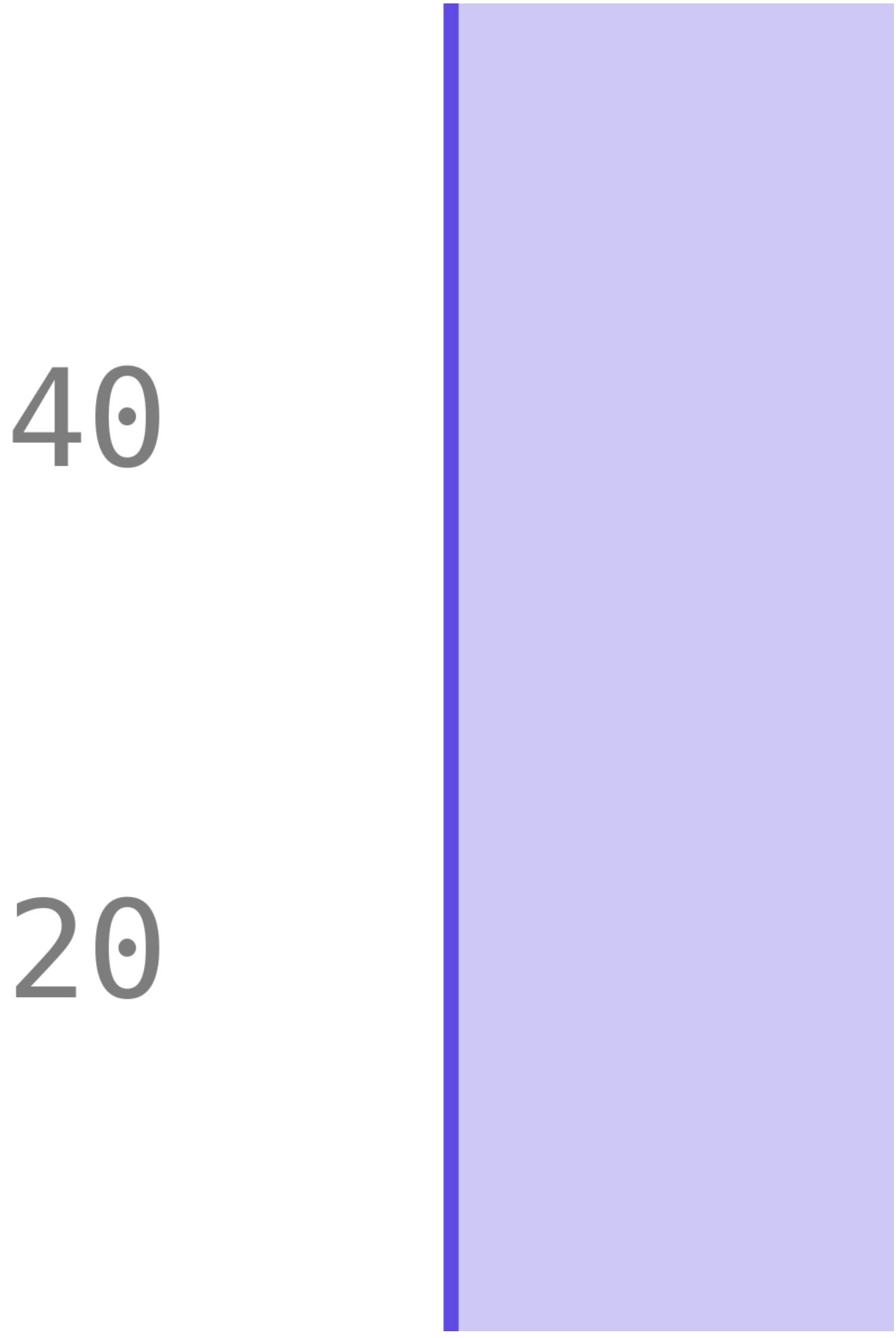
IFBench

Instruction f

76.5



60


40
20

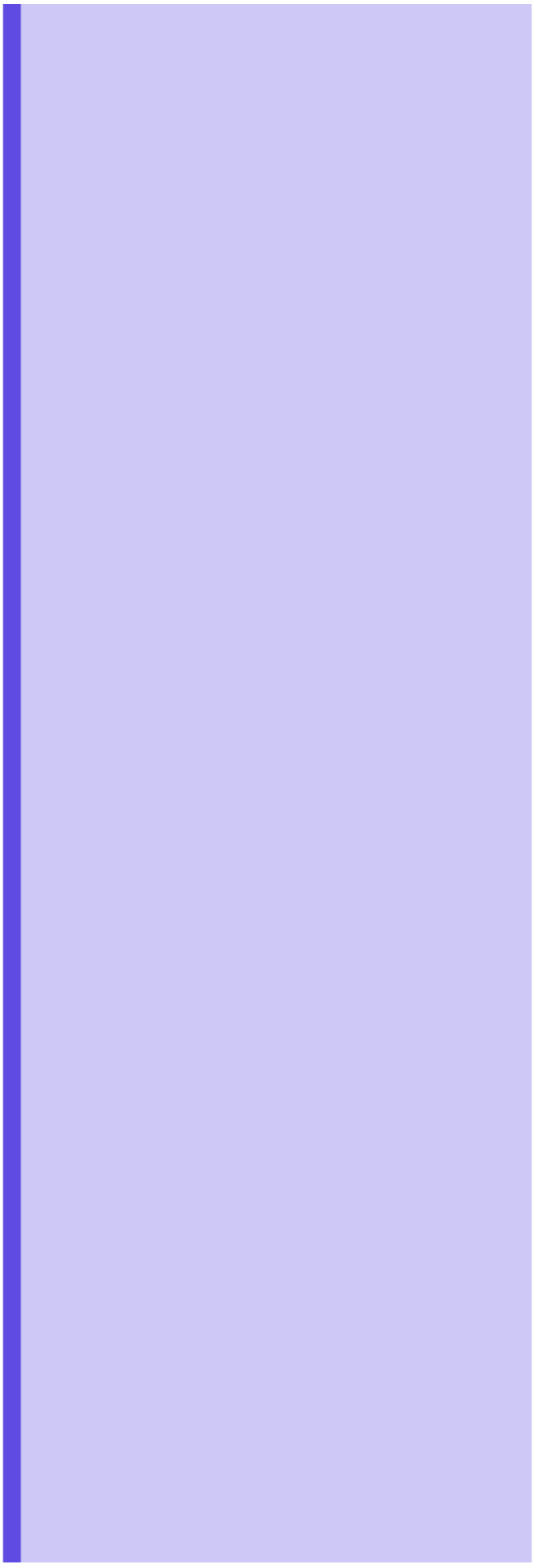


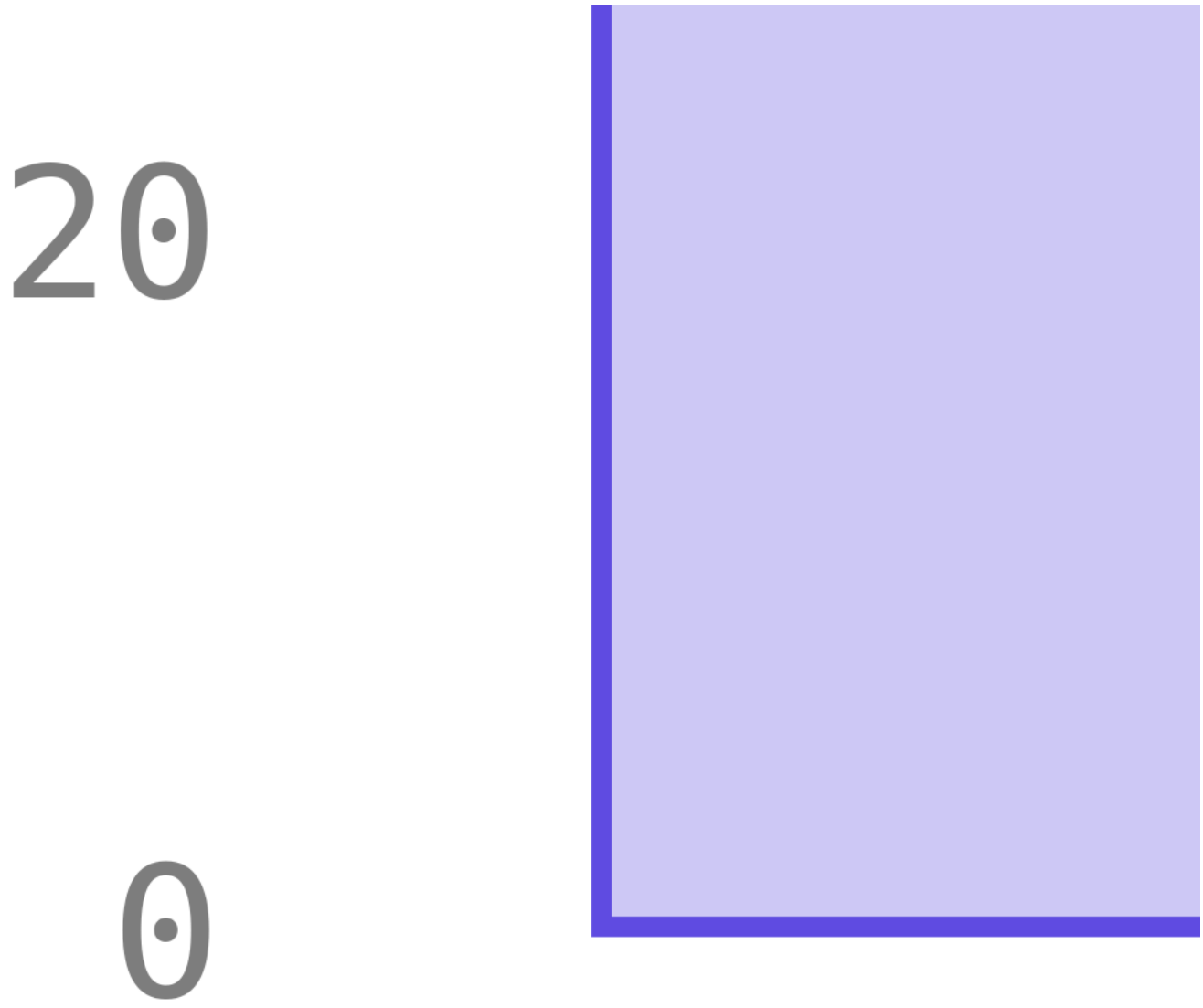
BrowseCo

Agentic sea

78.6

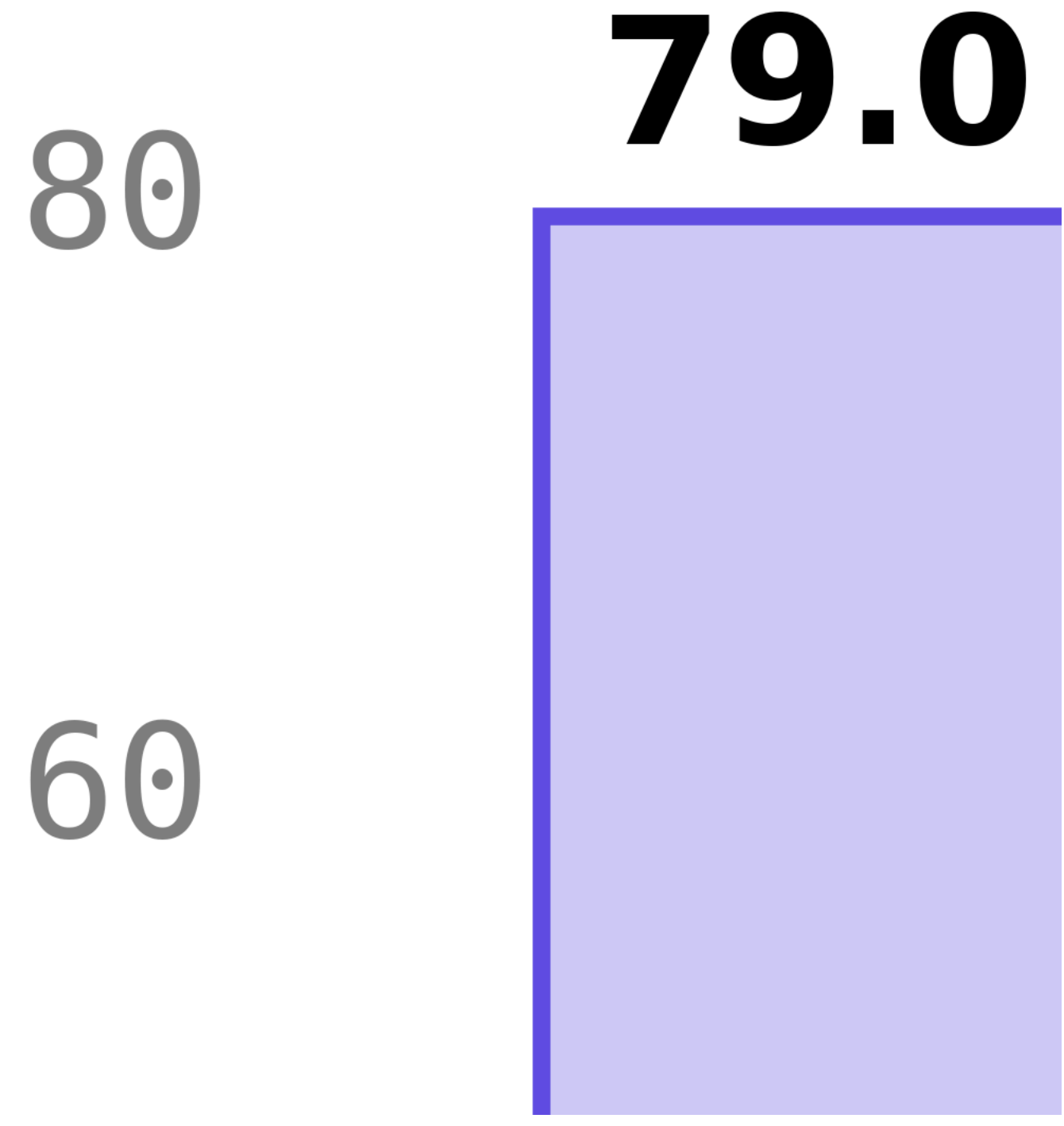


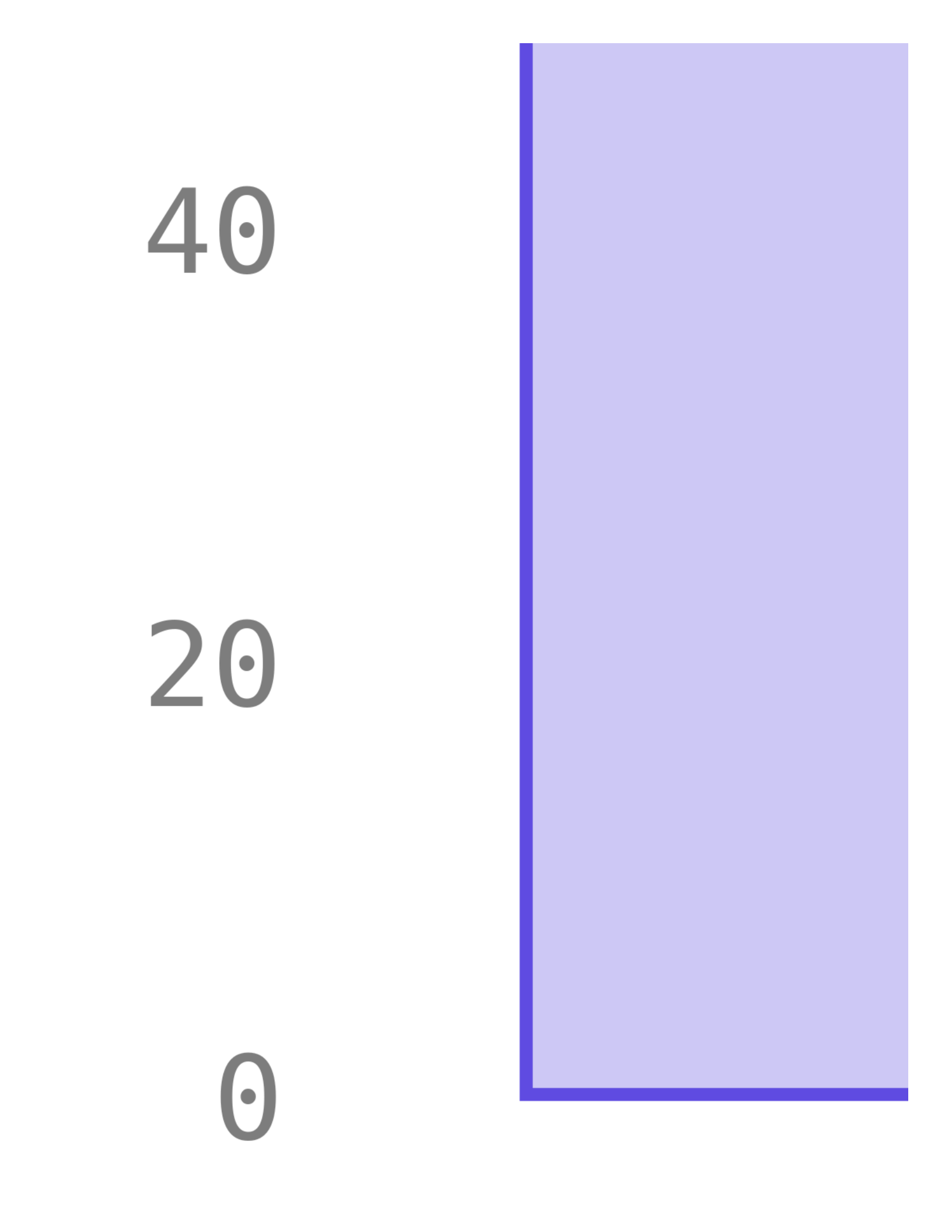

60
40



MMMU-Pr

Visual reasc




40
20
0

For more details, please refer to our blog post Qwen3.5.

Model Overview

- Type: Causal Language Model with Vision Encoder
- Training Stage: Pre-training & Post-training
- Language Model
  - Number of Parameters: 397B in total and 17B activated
  - Hidden Dimension: 4096
  - Token Embedding: 248320 (Padded)
  - Number of Layers: 60
    - Hidden Layout: 15 * (3 * (Gated DeltaNet -> MoE) -> 1 * (Gated Attention -> MoE))
  - Gated DeltaNet:
    - Number of Linear Attention Heads: 64 for V and 16 for QK
    - Head Dimension: 128
  - Gated Attention:
    - Number of Attention Heads: 32 for Q and 2 for KV
    - Head Dimension: 256
    - Rotary Position Embedding Dimension: 64
  - Mixture Of Experts
    - Number of Experts: 512
    - Number of Activated Experts: 10 Routed + 1 Shared
    - Expert Intermediate Dimension: 1024
  - LM Output: 248320 (Padded)
  - MTP: trained with multi-steps
- Context Length: 262,144 natively and extensible up to 1,010,000 tokens.

Benchmark Results

Language

	GPT5.2	Claude 4.5 Opus	Gemini-3 Pro	Qwen3-Max-Thinking	K2.5-1T-A32B	Qwen3.5-397B-A17B
Knowledge						
MMLU-Pro	87.4	89.5	89.8	85.7	87.1	87.8
MMLU-Redux	95.0	95.6	95.9	92.8	94.5	94.9
SuperGPQA	67.9	70.6	74.0	67.3	69.2	70.4
C-Eval	90.5	92.2	93.4	93.7	94.0	93.0
Instruction Following						
IFEval	94.8	90.9	93.5	93.4	93.9	92.6
IFBench	75.4	58.0	70.4	70.9	70.2	76.5
MultiChallenge	57.9	54.2	64.2	63.3	62.7	67.6
Long Context						
AA-LCR	72.7	74.0	70.7	68.7	70.0	68.7
LongBench v2	54.5	64.4	68.2	60.6	61.0	63.2
STEM						
GPQA	92.4	87.0	91.9	87.4	87.6	88.4
HLE	35.5	30.8	37.5	30.2	30.1	28.7
HLE-Verified[1]	43.3	38.8	48	37.6	–	37.6
Reasoning						
LiveCodeBench v6	87.7	84.8	90.7	85.9	85.0	83.6
HMMT Feb 25	99.4	92.9	97.3	98.0	95.4	94.8
HMMT Nov 25	100	93.3	93.3	94.7	91.1	92.7
IMOAnswerBench	86.3	84.0	83.3	83.9	81.8	80.9
AIME26	96.7	93.3	90.6	93.3	93.3	91.3
General Agent						
BFCL-V4	63.1	77.5	72.5	67.7	68.3	72.9
TAU2-Bench	87.1	91.6	85.4	84.6	77.0	86.7
VITA-Bench	38.2	56.3	51.6	40.9	41.9	49.7
DeepPlanning	44.6	33.9	23.3	28.7	14.5	34.3
Tool Decathlon	43.8	43.5	36.4	18.8	27.8	38.3
MCP-Mark	57.5	42.3	53.9	33.5	29.5	46.1
Search Agent[3]						
HLE w/ tool	45.5	43.4	45.8	49.8	50.2	48.3
BrowseComp	65.8	67.8	59.2	53.9	–/74.9	69.0/78.6
BrowseComp-zh	76.1	62.4	66.8	60.9	–	70.3
WideSearch	76.8	76.4	68.0	57.9	72.7	74.0
Seal-0	45.0	47.7	45.5	46.9	57.4	46.9
Multilingualism						
MMMLU	89.5	90.1	90.6	84.4	86.0	88.5

	GPT5.2	Claude 4.5 Opus	Gemini-3 Pro	Qwen3-Max-Thinking	K2.5-1T-A32B	Qwen3.5-397B-A17B
MMLU-ProX	83.7	85.7	87.7	78.5	82.3	84.7
NOVA-63	54.6	56.7	56.7	54.2	56.0	59.1
INCLUDE	87.5	86.2	90.5	82.3	83.3	85.6
Global PIQA	90.9	91.6	93.2	86.0	89.3	89.8
PolyMATH	62.5	79.0	81.6	64.7	43.1	73.3
WMT24++	78.8	79.7	80.7	77.6	77.6	78.9
MAXIFE	88.4	79.2	87.5	84.0	72.8	88.2
Coding Agent						
SWE-bench Verified	80.0	80.9	76.2	75.3	76.8	76.4
SWE-bench Multilingual	72.0	77.5	65.0	66.7	73.0	69.3
SecCodeBench	68.7	68.6	62.4	57.5	61.3	68.3
Terminal Bench 2	54.0	59.3	54.2	22.5	50.8	52.5

* HLE-Verified: a verified and revised version of Humanity's Last Exam (HLE), accompanied by a transparent, component-wise verification protocol and a fine-grained error taxonomy. We open-source the dataset at https://huggingface.co/datasets/skylenage/HLE-Verified.
* TAU2-Bench: we follow the official setup except for the airline domain, where all models are evaluated by applying the fixes proposed in the Claude Opus 4.5 system card.
* MCPMark: GitHub MCP server uses v0.30.3 from api.githubcopilot.com; Playwright tool responses are truncated at 32k tokens.
* Search Agent: most search agents built on our model adopt a simple context-folding strategy(256k): once the cumulative Tool Response length reaches a preset threshold, earlier Tool Responses are pruned from the history to keep the context within limits.
* BrowseComp: we tested two strategies, simple context-folding achieved a score of 69.0, while using the same discard-all strategy as DeepSeek-V3.2 and Kimi K2.5 achieved 78.6.
* WideSearch: we use a 256k context window without any context management.
* MMLU-ProX: we report the averaged accuracy on 29 languages.
* WMT24++: a harder subset of WMT24 after difficulty labeling and rebalancing; we report the averaged scores on 55 languages using XCOMET-XXL.
* MAXIFE: we report the accuracy on English + multilingual original prompts (totally 23 settings).
* Empty cells (--) indicate scores not yet available or not applicable.

Vision Language

	GPT5.2	Claude 4.5 Opus	Gemini-3 Pro	Qwen3-VL-235B-A22B	K2.5-1T-A32B	Qwen3.5-397B-A17B
STEM and Puzzle						
MMMU	86.7	80.7	87.2	80.6	84.3	85.0
MMMU-Pro	79.5	70.6	81.0	69.3	78.5	79.0
MathVision	83.0	74.3	86.6	74.6	84.2	88.6
Mathvista(mini)	83.1	80.0	87.9	85.8	90.1	90.3
We-Math	79.0	70.0	86.9	74.8	84.7	87.9
DynaMath	86.8	79.7	85.1	82.8	84.4	86.3
ZEROBench	9	3	10	4	9	12
ZEROBench_sub	33.2	28.4	39.0	28.4	33.5	41.0
BabyVision	34.4	14.2	49.7	22.2	36.5	52.3/43.3
General VQA						
RealWorldQA	83.3	77.0	83.3	81.3	81.0	83.9
MMStar	77.1	73.2	83.1	78.7	80.5	83.8
HallusionBench	65.2	64.1	68.6	66.7	69.8	71.4
$\text{MMBench}_{\text{EN-DEV-v1.1}}$	88.2	89.2	93.7	89.7	94.2	93.7
SimpleVQA	55.8	65.7	73.2	61.3	71.2	67.1
Text Recognition and Document Understanding						
OmniDocBench1.5	85.7	87.7	88.5	84.5	88.8	90.8
CharXiv(RQ)	82.1	68.5	81.4	66.1	77.5	80.8
MMLongBench-Doc	--	61.9	60.5	56.2	58.5	61.5
CC-OCR	70.3	76.9	79.0	81.5	79.7	82.0
AI2D_TEST	92.2	87.7	94.1	89.2	90.8	93.9
OCRBench	80.7	85.8	90.4	87.5	92.3	93.1
Spatial Intelligence						
ERQA	59.8	46.8	70.5	52.5	--	67.5

	GPT5.2	Claude 4.5 Opus	Gemini-3 Pro	Qwen3-VL-235B-A22B	K2.5-1T-A32B	Qwen3.5-397B-A17B
CountBench	91.9	90.6	97.3	93.7	94.1	97.2
RefCOCO(avg)	--	--	84.1	91.1	87.8	92.3
ODInW13	--	--	46.3	43.2	--	47.0
EmbSpatialBench	81.3	75.7	61.2	84.3	77.4	84.5
RefSpatialBench	--	--	65.5	69.9	--	73.6
LingoQA	68.8	78.8	72.8	66.8	68.2	81.6
V*	75.9	67.0	88.0	85.9	77.0	95.8/91.1
Hypersim	--	--	--	11.0	--	12.5
SUNRGBD	--	--	--	34.9	--	38.3
Nuscene	--	--	--	13.9	--	16.0
Video Understanding						
VideoMME$_{(w\ sub.)}$	86	77.6	88.4	83.8	87.4	87.5
VideoMME$_{(w/o\ sub.)}$	85.8	81.4	87.7	79.0	83.2	83.7
VideoMMMU	85.9	84.4	87.6	80.0	86.6	84.7
MLVU (M-Avg)	85.6	81.7	83.0	83.8	85.0	86.7
MVBench	78.1	67.2	74.1	75.2	73.5	77.6
LVBench	73.7	57.3	76.2	63.6	75.9	75.5
MMVU	80.8	77.3	77.5	71.1	80.4	75.4
Visual Agent						
ScreenSpot Pro	--	45.7	72.7	62.0	--	65.6
OSWorld-Verified	38.2	66.3	--	38.1	63.3	62.2
AndroidWorld	--	--	--	63.7	--	66.8
Medical VQA						
SLAKE	76.9	76.4	81.3	72.5	81.6	79.9
PMC-VQA	58.9	59.9	62.3	56.1	63.3	64.2
MedXpertQA-MM	73.3	63.6	76.0	47.6	65.3	70.0

* MathVision： our model's score is evaluated using a fixed prompt, e.g., "Please reason step by step, and put your final answer within \boxed{}." For other models, we report the higher score between runs with and without the \boxed{} formatting.
* BabyVision: our model's score is reported with CI (Code Interpreter) enabled; without CI, the result is 43.3.
* V*: our model's score is reported with CI (Code Interpreter) enabled; without CI, the result is 91.1.
* Empty cells (--) indicate scores not yet available or not applicable.

Quickstart

> [!Important] Qwen3.5 models operate in thinking mode by default, generating thinking content signified by `<think>\n...</think>\n\n` before producing the final responses. To disable thinking content and obtain direct response, refer to the examples here.

For streamlined integration, we recommend using Qwen3.5 via APIs. Below is a guide to use Qwen3.5 via OpenAI-compatible API.

Serving Qwen3.5

Qwen3.5 can be served via APIs with popular inference frameworks. In the following, we show example commands to launch OpenAI-Compatible API servers for Qwen3.5 models.

> [!Important] Inference efficiency and throughput vary significantly across frameworks. We recommend using the latest framework versions to ensure optimal performance and compatibility. For production workloads or high-throughput scenarios, dedicated serving engines such as SGLang, KTransformers or vLLM are strongly recommended.

> [!Important] The model has a default context length of 262,144 tokens. If you encounter out-of-memory (OOM) errors, consider reducing the context window. However, because Qwen3.5 leverages extended context for complex tasks, we advise maintaining a context length of at least 128K tokens to preserve thinking capabilities.

SGLang

SGLang is a fast serving framework for large language models and vision language models. SGLang from the main branch of the open-source repository is required for Qwen3.5, which can be installed using the following command in a fresh environment:

```
uv pip install 'git+https://github.com/sgl-project/sglang.git#subdirectory=python&egg=sglang[all]'
```

See its documentation for more details.

The following will create API endpoints at `http://localhost:8000/v1`:

- **Standard Version**: The following command can be used to create an API endpoint with maximum context length 262,144 tokens using tensor parallel on 8 GPUs.

  ```
  python -m sglang.launch_server --model-path Qwen/Qwen3.5-397B-A17B --port 8000 --tp-size 8 --mem-fraction-static 0.8 --context-length 262144 --reasoning-parser qwen3
  ```

- **Tool Use**: To support tool use, you can use the following command.

  ```
  python -m sglang.launch_server --model-path Qwen/Qwen3.5-397B-A17B --port 8000 --tp-size 8 --mem-fraction-static 0.8 --context-length 262144 --reasoning-parser qwen3 --tool-call-parser qwen3_coder
  ```

- **Multi-Token Prediction (MTP)**: The following command is recommended for MTP:

  ```
  python -m sglang.launch_server --model-path Qwen/Qwen3.5-397B-A17B --port 8000 --tp-size 8 --mem-fraction-static 0.8 --context-length 262144 --reasoning-parser qwen3 --speculative-algo NEXTN --speculative-num-steps 3 --speculative-eagle-topk 1 --speculative-num-draft-tokens 4
  ```

vLLM

vLLM is a high-throughput and memory-efficient inference and serving engine for LLMs. vLLM from the main branch of the open-source repository is required for Qwen3.5, which can be installed using the following command in a fresh environment:

```
uv pip install vllm --torch-backend=auto --extra-index-url https://wheels.vllm.ai/nightly
```

See its documentation for more details.

For detailed Qwen3.5 usage guide, see the vLLM Qwen3.5 recipe.

The following will create API endpoints at `http://localhost:8000/v1`:

- **Standard Version**: The following command can be used to create an API endpoint with maximum context length 262,144 tokens using tensor parallel on 8 GPUs.

  ```
  vllm serve Qwen/Qwen3.5-397B-A17B --port 8000 --tensor-parallel-size 8 --max-model-len 262144 --reasoning-parser qwen3
  ```

- **Tool Call**: To support tool use, you can use the following command.

  ```
  vllm serve Qwen/Qwen3.5-397B-A17B --port 8000 --tensor-parallel-size 8 --max-model-len 262144 --reasoning-parser qwen3 --enable-auto-tool-choice --tool-call-parser qwen3_coder
  ```

- **Multi-Token Prediction (MTP)**: The following command is recommended for MTP:

  ```
  vllm serve Qwen/Qwen3.5-397B-A17B --port 8000 --tensor-parallel-size 8 --max-model-len 262144 --reasoning-parser qwen3 --speculative-config '{"method":"qwen3_next_mtp","num_speculative_tokens":2}'
  ```

- **Text-Only**: The following command skips the vision encoder and multimodal profiling to free up memory for additional KV cache:

  ```
  vllm serve Qwen/Qwen3.5-397B-A17B --port 8000 --tensor-parallel-size 8 --max-model-len 262144 --reasoning-parser qwen3 --language-model-only
  ```

KTransformers

KTransformers is a flexible framework for experiencing cutting-edge LLM inference optimizations with CPU-GPU heterogeneous computing. For running Qwen3.5 with KTransformers, see the KTransformers Deployment Guide.

Hugging Face Transformers

Hugging Face Transformers contains a *lightweight* server which can be used for quick testing and moderate load deployment. The latest `transformers` is required for Qwen3.5:

```
pip install "transformers[serving] @ git+https://github.com/huggingface/transformers.git@main"
```

See its documentation for more details. Please also make sure torchvision and pillow are installed.

Then, run `transformers serve` to launch a server with API endpoints at `http://localhost:8000/v1`; it will place the model on accelerators if available:

```
transformers serve --force-model Qwen/Qwen3.5-397B-A17B --port 8000 --continuous-batching
```

Using Qwen3.5 via the Chat Completions API

The chat completions API is accessible via standard HTTP requests or OpenAI SDKs. Here, we show examples using the OpenAI Python SDK.

Before starting, make sure it is installed and the API key and the API base URL is configured, e.g.:

```
pip install -U openai

# Set the following accordingly
export OPENAI_BASE_URL="http://localhost:8000/v1"
export OPENAI_API_KEY="EMPTY"
```

> [!Tip] We recommend using the following set of sampling parameters for generation
>
> - Thinking mode: `temperature=0.6, top_p=0.95, top_k=20, min_p=0.0, presence_penalty=0.0, repetition_penalty=1.0`
> - Instruct (or non-thinking) mode: `temperature=0.7, top_p=0.8, top_k=20, min_p=0.0, presence_penalty=1.5, repetition_penalty=1.0`
>
> Please note that the support for sampling parameters varies according to inference frameworks.

Text-Only Input

```
from openai import OpenAI
# Configured by environment variables
client = OpenAI()

messages = [
    {"role": "user", "content": "Type \"I love Qwen3.5\" backwards"},
]

chat_response = client.chat.completions.create(
    model="Qwen/Qwen3.5-397B-A17B",
    messages=messages,
    max_tokens=81920,
    temperature=0.6,
    top_p=0.95,
    extra_body={
        "top_k": 20,
    },
)
print("Chat response:", chat_response)
```

Image Input

```
from openai import OpenAI
# Configured by environment variables
client = OpenAI()

messages = [
    {
        "role": "user",
        "content": [
            {
                "type": "image_url",
                "image_url": {
                    "url": "https://qianwen-res.oss-accelerate.aliyuncs.com/Qwen3.5/demo/CI_Demo/mathv-1327.jpg"
                }
            },
            {
                "type": "text",
                "text": "The centres of the four illustrated circles are in the corners of the square. The two big circles touch each other
        and also the two little circles. With which factor do you have to multiply the radii of the little circles to obtain the radius of
        the big circles?\nChoices:\n(A) $\\frac{2}{9}$\n(B) $\\sqrt{5}$\n(C) $0.8 \\cdot \\pi$\n(D) 2.5\n(E) $1+\\sqrt{2}$"
            }
        ]
    }
]

chat_response = client.chat.completions.create(
    model="Qwen/Qwen3.5-397B-A17B",
    messages=messages,
    max_tokens=81920,
    temperature=0.6,
    top_p=0.95,
    extra_body={
        "top_k": 20,
    },
)
print("Chat response:", chat_response)
```

Video Input

```
from openai import OpenAI
# Configured by environment variables
client = OpenAI()

messages = [
    {
        "role": "user",
        "content": [
            {
                "type": "video_url",
                "video_url": {
                    "url": "https://qianwen-res.oss-accelerate.aliyuncs.com/Qwen3.5/demo/video/N1cdUjctpG8.mp4"
                }
            },
            {
                "type": "text",
                "text": "How many porcelain jars were discovered in the niches located in the primary chamber of the tomb?"
            }
        ]
    }
]

# When vLLM is launched with `--media-io-kwargs '{"video": {"num_frames": -1}}'`,
# video frame sampling can be configured via `extra_body` (e.g., by setting `fps`).
# This feature is currently supported only in vLLM.
#
# By default, `fps=2` and `do_sample_frames=True`.
```

```
# With `do_sample_frames=True`, you can customize the `fps` value to set your desired video sampling rate.
chat_response = client.chat.completions.create(
    model="Qwen/Qwen3.5-397B-A17B",
    messages=messages,
    max_tokens=81920,
    temperature=0.6,
    top_p=0.95,
    extra_body={
        "top_k": 20,
        "mm_processor_kwargs": {"fps": 2, "do_sample_frames": True},
    },
)

print("Chat response:", chat_response)
```

Instruct (or Non-Thinking) Mode

> [!Important] Qwen3.5 does not officially support the soft switch of Qwen3, i.e., `/think` and `/nothink`.

Qwen3.5 will think by default before response. You can obtain direct response from the model without thinking by configuring the API parameters. For example,

```
from openai import OpenAI
# Configured by environment variables
client = OpenAI()

messages = [
    {
        "role": "user",
        "content": [
            {
                "type": "image_url",
                "image_url": {
                    "url": "https://qianwen-res.oss-accelerate.aliyuncs.com/Qwen3.5/demo/RealWorld/RealWorld-04.png"
                }
            },
            {
                "type": "text",
                "text": "Where is this?"
            }
        ]
    }
]

chat_response = client.chat.completions.create(
    model="Qwen/Qwen3.5-397B-A17B",
    messages=messages,
    max_tokens=32768,
    temperature=0.7,
    top_p=0.8,
    presence_penalty=1.5,
    extra_body={
        "top_k": 20,
        "chat_template_kwargs": {"enable_thinking": False},
    },
)
print("Chat response:", chat_response)
```

> [!Note] If you are using APIs from Alibaba Cloud Model Studio, in addition to changing `model`, please use `"enable_thinking": False` instead of `"chat_template_kwargs": {"enable_thinking": False}`.

Agentic Usage

Qwen3.5 excels in tool calling capabilities.

Qwen-Agent

We recommend using Qwen-Agent to quickly build Agent applications with Qwen3.5.

To define the available tools, you can use the MCP configuration file, use the integrated tool of Qwen-Agent, or integrate other tools by yourself.

```
import os
from qwen_agent.agents import Assistant

# Define LLM
# Using Alibaba Cloud Model Studio
llm_cfg = {
    # Use the OpenAI-compatible model service provided by DashScope:
    'model': 'Qwen3.5-397B-A17B',
    'model_type': 'qwenvl_oai',
    'model_server': 'https://dashscope.aliyuncs.com/compatible-mode/v1',
    'api_key': os.getenv('DASHSCOPE_API_KEY'),

    'generate_cfg': {
        'use_raw_api': True,
        # When using Dash Scope OAI API, pass the parameter of whether to enable thinking mode in this way
        'extra_body': {
            'enable_thinking': True
```

```
        },
    },
}

# Using OpenAI-compatible API endpoint.
# functionality of the deployment frameworks and let Qwen-Agent automate the related operations.
#
# llm_cfg = {
#     # Use your own model service compatible with OpenAI API by vLLM/SGLang:
#     'model': 'Qwen/Qwen3.5-397B-A17B',
#     'model_type': 'qwenvl_oai',
#     'model_server': 'http://localhost:8000/v1',  # api_base
#     'api_key': 'EMPTY',
#
#     'generate_cfg': {
#         'use_raw_api': True,
#         # When using vLLM/SGLang OAI API, pass the parameter of whether to enable thinking mode in this way
#         'extra_body': {
#             'chat_template_kwargs': {'enable_thinking': True}
#         },
#     },
# }

# Define Tools
tools = [
    {'mcpServers': {  # You can specify the MCP configuration file
            "filesystem": {
                "command": "npx",
                "args": ["-y", "@modelcontextprotocol/server-filesystem", "/Users/xxxx/Desktop"]
            }
        }
    }
]

# Define Agent
bot = Assistant(llm=llm_cfg, function_list=tools)

# Streaming generation
messages = [{'role': 'user', 'content': 'Help me organize my desktop.'}]
for responses in bot.run(messages=messages):
    pass
print(responses)

# Streaming generation
messages = [{'role': 'user', 'content': 'Develop a dog website and save it on the desktop'}]
for responses in bot.run(messages=messages):
    pass
print(responses)
```

Qwen Code

Qwen Code is an open-source AI agent for the terminal, optimized for Qwen models. It helps you understand large codebases, automate tedious work, and ship faster.

For more information, please refer to Qwen Code.

Processing Ultra-Long Texts

Qwen3.5 natively supports context lengths of up to 262,144 tokens. For long-horizon tasks where the total length (including both input and output) exceeds this limit, we recommend using RoPE scaling techniques to handle long texts effectively., e.g., YaRN.

YaRN is currently supported by several inference frameworks, e.g., `transformers`, `vllm`, `ktransformers` and `sglang`. In general, there are two approaches to enabling YaRN for supported frameworks:

- Modifying the model configuration file: In the `config.json` file, change the `rope_parameters` fields in `text_config` to:

  ```
  {
      "mrope_interleaved": true,
      "mrope_section": [
          11,
          11,
          10
      ],
      "rope_type": "yarn",
      "rope_theta": 10000000,
      "partial_rotary_factor": 0.25,
      "factor": 4.0,
      "original_max_position_embeddings": 262144,
  }
  ```

- Passing command line arguments:

  For `vllm`, you can use

  ```
  VLLM_ALLOW_LONG_MAX_MODEL_LEN=1 vllm serve ... --hf-overrides '{"text_config": {"rope_parameters": {"mrope_interleaved": true,
  "mrope_section": [11, 11, 10], "rope_type": "yarn", "rope_theta": 10000000, "partial_rotary_factor": 0.25, "factor": 4.0,
  "original_max_position_embeddings": 262144}}}' --max-model-len 1010000
  ```

  For `sglang` and `ktransformers`, you can use

```
SGLANG_ALLOW_OVERWRITE_LONGER_CONTEXT_LEN=1 python -m sglang.launch_server ... --json-model-override-args '{"text_config":
{"rope_parameters": {"mrope_interleaved": true, "mrope_section": [11, 11, 10], "rope_type": "yarn", "rope_theta": 10000000,
"partial_rotary_factor": 0.25, "factor": 4.0, "original_max_position_embeddings": 262144}}}' --context-length 1010000
```

[!NOTE] All the notable open-source frameworks implement static YaRN, which means the scaling factor remains constant regardless of input length, **potentially impacting performance on shorter texts.** We advise modifying the `rope_parameters` configuration only when processing long contexts is required. It is also recommended to modify the `factor` as needed. For example, if the typical context length for your application is 524,288 tokens, it would be better to set `factor` as 2.0.

Best Practices

To achieve optimal performance, we recommend the following settings:

1. **Sampling Parameters**:
   - We suggest using `Temperature=0.6`, `TopP=0.95`, `TopK=20`, and `MinP=0` for thinking mode and using `Temperature=0.7`, `TopP=0.8`, `TopK=20`, and `MinP=0` for non-thinking mode.
   - For supported frameworks, you can adjust the `presence_penalty` parameter between 0 and 2 to reduce endless repetitions. However, using a higher value may occasionally result in language mixing and a slight decrease in model performance.

2. **Adequate Output Length**: We recommend using an output length of 32,768 tokens for most queries. For benchmarking on highly complex problems, such as those found in math and programming competitions, we suggest setting the max output length to 81,920 tokens. This provides the model with sufficient space to generate detailed and comprehensive responses, thereby enhancing its overall performance.

3. **Standardize Output Format**: We recommend using prompts to standardize model outputs when benchmarking.
   - **Math Problems**: Include "Please reason step by step, and put your final answer within \boxed{}." in the prompt.
   - **Multiple-Choice Questions**: Add the following JSON structure to the prompt to standardize responses: "Please show your choice in the `answer` field with only the choice letter, e.g., `"answer": "C"`."

4. **No Thinking Content in History**: In multi-turn conversations, the historical model output should only include the final output part and does not need to include the thinking content. It is implemented in the provided chat template in Jinja2. However, for frameworks that do not directly use the Jinja2 chat template, it is up to the developers to ensure that the best practice is followed.

5. **Long Video Understanding**: To optimize inference efficiency for plain text and images, the `size` parameter in the released `video_preprocessor_config.json` is conservatively configured. It is recommended to set the `longest_edge` parameter in the video_preprocessor_config file to 469,762,048 (corresponding to 224k video tokens) to enable higher frame-rate sampling for hour-scale videos and thereby achieve superior performance. For example,

   ```
   {"longest_edge": 469762048, "shortest_edge": 4096}
   ```

   Alternatively, override the default values via engine startup parameters. For implementation details, refer to: vLLM / SGLang.

Citation

If you find our work helpful, feel free to give us a cite.

```
@misc{qwen3.5,
    title  = {{Qwen3.5}: Towards Native Multimodal Agents},
    author = {{Qwen Team}},
    month  = {February},
    year   = {2026},
    url    = {https://qwen.ai/blog?id=qwen3.5}
}
```